Практическая работа: Data Placement Map - “Курсатор”

ЭТАП 1 - Постановка задачи

Кейс: образовательная платформа «Курсатор» - онлайн-обучение с курсами, тестами, форумом, авторами, модераторами и аналитикой.

Задача: распределить 9 сущностей/потоков между хранилищами (PostgreSQL, Redis, MongoDB, ClickHouse), указав операции, нагрузку, обоснование, источник истины и спорные моменты.

ЭТАП 2. Анализ сущностей

Сущность	Часто читают или пишут?	Нужны транзакции?	Нужны JOIN / связи?	Объём данных?	Требования к задержке?	Гибкость схемы?
1. Пользователи	Чтение часто, запись редко	ДА (регистрация, смена пароля, прав)	ДА (роли, курсы, прогресс)	Средний (десятки-сотни тысяч)	Низкие (быстрый вход)	Нет (стабильная схема)
2. Сессии и токены	ОЧЕНЬ часто чтение	НЕТ	НЕТ	Большой, но с TTL	Очень низкие (мс)	Нет
3. Курсы и уроки	Чтение часто, запись редко	ДА (публикация)	ДА (вложенность курс→модуль→урок)	Средний	Средние	ДА (разный набор полей)
4. Прогресс по курсам	ОЧЕНЬ часто чтение и запись	Желательно	ДА (user–course–lesson)	Большой (миллионы)	Низкие	Нет
5. Поиск по курсам	Часто чтение	НЕТ	Частично	Средний	Низкие	Нет

6. Форум / обсуждения	Чтение и запись средне	Желательно	ДА (древовидная структура)	Средний-большой	Средние	Частично
7. Логи действий	ОЧЕНЬ часто запись, редко чтение	НЕТ	НЕТ	Огромный (миллиарды)	Высокие (асинхронно)	Нет
8. Аналитика и отчёты	Редко чтение, тяжело	НЕТ	ДА (агрегации)	Огромный	Высокие (минуты)	Нет
9. Рекомендации курсов	Часто чтение	НЕТ	ДА (user–course связи)	Средний	Низкие	Нет

Вопросы для обсуждения:

- Источник истины: пользователи, курсы, прогресс - PostgreSQL/MongoDB; логи - ClickHouse.
- Кэшировать: сессии (Redis, TTL 30 мин), рекомендации (Redis, TTL 1 час), топ курсов (Redis, TTL 10 мин).
- Для аналитики: производные данные из логов и прогресса - ClickHouse.
- ACID: пользователи, прогресс, форум. Eventual consistency: кэш, рекомендации, поиск.

ЭТАП 3. Data Placement Map

Сущность / поток	Операции	Характер нагрузки	Хранилище	Обоснование (2–3 фразы)	Источник истины?	Кэш?	Спорные моменты
1. Пользователи	регистрация, вход, редактирование, проверка прав	Чтение часто, запись редко, нужны ACID	PostgreSQL	Нужна целостность данных, связи с ролями, транзакции при смене пароля/прав. Реляционн	ДА	Redis (профиль пользователя, TTL 15 мин)	Можно ли часть полей профиля хранить в MongoDB из-за гибкости?

				ая модель идеальна.			
2. Сессии и токены	создание, проверка, истечение по TTL	Очень частое чтение, короткая жизнь	Redis	Нативная поддержка TTL, скорость доступа <1 мс, идеален для сессий. Данные эфемерные.	ДА	— (само является кэшем)	Дублировать ли сессию в PostgreSQL для аудита?
3. Курсы и уроки	создание, редактирование, публикация	Чтение часто, запись редко, вложенная структура	MongoDB	Вложенная структура (курс→модули→уроки→материалы), разный набор полей у курсов — документная модель подходит лучше реляционной.	ДА	Redis (популярные курсы, TTL 10 мин)	Часть метаданных (цена, автор) лучше в PostgreSQL для транзакций?
4. Прогресс по курсам	отметка уроков, результаты тестов	Очень частые чтение и запись, много записей	PostgreSQL	Нужны транзакции (зачёт теста = обновление прогресса + начисление баллов), связи	ДА	Redis (текущий прогресс пользователя, TTL 5 мин)	При очень большом объёме — не лучше ли ClickHouse?

				user–course–lesson.			
5. Поиск по курсам	полнотекстовый поиск, фильтры, опечатки	Часто чтение	PostgreSQL (FTS)	Встроенный полнотекстовый поиск (tsvector/tsquery), индексы GIN, ранжирование. Не требует отдельного движка.	НЕТ (производное из MongoDB)	—	Стоит ли использовать Elasticsearch вместо встроенного FTS?
6. Форум / обсуждения	темы, ответы, лайки	Чтение и запись средне, древовидная структура	PostgreSQL	Recursive CTE для деревьев, транзакции для лайков/модерации, moderate volume — реляционная БД справится.	ДА	Redis (счётчики лайков, горячие темы)	При росте — не перейти ли на MongoDB для гибкости структуры?
7. Логи действий	клики, просмотры, старт/завершение тестов	Огромный объём записи, редкие чтения	ClickHouse	Колоночное хранение, высокая скорость записи, идеален для событийных потоков и последующей аналитики.	ДА	—	Нужен ли буфер (Kafka) перед ClickHouse?

8. Аналитика и отчёты	агрегаты, воронки, статистика	Редкое чтение, тяжёлые запросы	ClickHouse	Быстрые агрегации по большим объёмам, материализованные представления, исторические данные.	НЕТ (производное из логов и прогресса)	—	Не дублировать ли данные из PostgreSQL в ClickHouse?
9. Рекомендации курсов	«похожие курсы», «популярное у похожих»	Часто чтение	Redis	Быстрое чтение готовых рекомендаций, структуры типа Sorted Set для ранжирования.	НЕТ (производное)	— (само является кэшем)	Где считать рекомендации — в PostgreSQL или отдельным сервисом?

ЭТАП 4. Синхронизация и потоки данных

[Пользователи] → [PostgreSQL] → [CDC / триггеры] → [Redis (кэш профиля)]
[Курсы и уроки] → [MongoDB] → [Change Streams] → [PostgreSQL FTS-индекс (поиск)]
[Курсы и уроки] → [MongoDB] → [Change Streams] → [Redis (кэш популярных курсов)]
[Прогресс по курсам] → [PostgreSQL] → [Триггеры / очередь] → [Redis (кэш текущего прогресса)]
[Прогресс по курсам] → [PostgreSQL] → [ETL-джоба (раз в час)] → [ClickHouse (аналитика)]
[Логи действий] → [ClickHouse] → [Материализованные представления] → [ClickHouse (агрегаты для отчётов)]
[Логи действий] → [ClickHouse] → [ETL-джоба] → [Redis (популярные курсы для рекомендаций)]
[Рекомендации] → [Сервис рекомендаций (раз в сутки)] → [Redis (готовые рекомендации)]
[Форум] → [PostgreSQL] → [Триггеры] → [Redis (счётчики лайков, горячие темы)]

Ключевые принципы:

- Источник истины всегда один (PostgreSQL / MongoDB / ClickHouse).

- Производные хранилища обновляются через: триггеры, Change Streams (MongoDB), CDC, ETL-джобы, очереди.
- Кэш (Redis) имеет TTL и может быть пересоздан из источника в любой момент.